Yault Platform — Security Assessment Report

Field	Detail
Project	Yault — Decentralized Asset Inheritance & Estate Planning Platform
Version	Mainnet-candidate (February 2026)
Assessment Date	2026-02-26
Assessor	AI-assisted security review (Claude, Anthropic)
Methodology	Static code review, runtime debugging, architecture analysis

Table of Contents

1. Executive Summary
2. Assessment Scope
3. Current Security Posture
4. Authentication & Authorization
5. Data Layer Security
6. External Service Resilience
7. Smart Contract Security
8. Operational Controls
9. Areas for Improvement
10. Pre-Launch Recommendations
11. Scope Limitations & Disclaimer

Executive Summary

This report presents the **current security posture** of the Yault platform as of February 2026.

Yault is a decentralized asset inheritance and estate planning platform consisting of a Node.js backend API, four Solidity smart contracts, Arweave permanent storage, and a web frontend. The system manages user vaults (ERC-4626), share escrows, release attestations, and RWA credential NFT delivery.

Overall Assessment: **Production-Ready with Caveats**

The platform implements defense-in-depth security across its backend, with proper authentication, authorization, rate limiting, input validation, and operational controls. All identified exploitable vulnerabilities have been remediated.

Before mainnet launch, the following should be completed:

- Formal third-party smart contract audit
- Migration from SQLite (dev) to PostgreSQL (production)
- HSM key management for the relayer wallet

Security Controls Summary

Control	Status
Dual-method authentication	✅ Implemented
Address normalization	✅ Consistent
SQL parameterization	✅ Correct patterns

Rate limiting	✅ Per-endpoint
Multi-sig admin approvals	✅ 2-of-N for all admin ops
Arweave multi-gateway failover	✅ Promise.any racing
Attestation idempotency	✅ On-chain pre-check
Concurrent delivery protection	✅ Process-level lock
Escrow calculation integrity	✅ Explicit null checks
Audit logging	✅ Multi-sig operations

Assessment Scope

In-Scope Components

Layer	Path / Component	Description
Backend API	`server/api/**`	All REST endpoints (vault, trigger, claim, admin)
Services	`server/services/**`	Arweave storage, scheduler, background jobs
Authentication	`server/middleware/`, `server/api/admin/`	Dual-auth middleware, admin wallet auth
Database	`server/db.js`	sql.js (SQLite via WebAssembly) data layer
Smart Contracts	`contracts/src/ReleaseAttestation.sol`	On-chain attestation (oracle + fallback)
Smart Contracts	`contracts/src/YaultVault.sol`	ERC-4626 yield vault for asset management
Smart Contracts	`contracts/src/VaultShareEscrow.sol`	Per-recipient share escrow with claim mechanism
Smart Contracts	`contracts/src/YaultPathClaim.sol`	USDC path-based claim pool
Middleware	`server/middleware/multisig.js`	Multi-signature approval system
Configuration	`server/config/`, `server/index.js`	Route mounting, rate limiting, env config

Out-of-Scope

- Frontend application (`webapp/`)
- Third-party npm dependencies & OpenZeppelin libraries
- Infrastructure and deployment configuration
- Formal smart contract verification / fuzzing

Current Security Posture

3.1 Authentication Architecture

The platform uses a **dual-method authentication system**:

Method	Mechanism	Use Case
Wallet Signature	`Authorization: EVM <challengeId>:<signature>`	Primary user auth, admin multi-sig
Session Token	`X-Admin-Session` header	Ops portal admin sessions
Legacy Token	`X-Admin-Token` header (timing-safe comparison)	Backward-compatible admin auth

Key security properties:

- All address comparisons use `normalizeAddr()` to strip `0x` prefix and lowercase before comparison, preventing case-sensitivity bypass.
- Admin token comparison uses `crypto.timingSafeEqual()` to prevent timing attacks.
- Wallet signature verification uses `verifySignature()` with challenge-response to prevent replay attacks.

3.2 Authorization Model

All endpoints enforce proper authorization:

Endpoint Category	Auth Required	Authorization Check
Vault balance	✅	Caller must own the queried address
Escrow balance	✅	Caller must be plan owner or authorized recipient
Activity feed	✅	Filtered to caller's own activities
Admin operations	✅	Admin wallet + multi-sig approval (2-of-N)
Path-claim	✅	Dual-auth middleware applied
Binding create/delete	✅	Caller must own the binding

3.3 Rate Limiting

Endpoint-specific rate limiting is applied to all sensitive routes:

Endpoint Group	Limit	Rationale
`mnemonic-hash`	30 req/min	Prevent brute-force hash enumeration
`release/distribute`	20 req/min	Prevent Arweave upload spam
`invite/accept`	30 req/min	Prevent invite flooding
`admin/session`	10 req/min	Prevent admin credential brute-force

3.4 Input Validation & SQL Safety

All database queries use **parameterized statements** via the sql.js `prepare/bind/step` pattern:

```
const stmt = innerDb.prepare('SELECT data FROM "table" WHERE id = ?');
stmt.bind([paramValue]);
// ...
stmt.free();
```

This prevents SQL injection regardless of the underlying database engine.

Authentication & Authorization

4.1 Multi-Signature Admin Approvals

All sensitive admin operations require **2-of-N multi-signature approval** from authorized admin wallets:

Action	Required Approvals	Expiry	Category
Emergency Release	2	1 hour	Critical

Trigger Abort	2	1 hour	Critical
Trigger Resume	2	1 hour	Critical
Legal Confirmation	2	1 hour	Critical
KYC Review	2	4 hours	Sensitive
User Role Change	2	4 hours	Sensitive
Authority Verification	2	4 hours	Sensitive
Force Redeliver	2	4 hours	Sensitive

Multi-sig flow:

1. Admin A calls the endpoint → creates a pending approval (HTTP 202), returns `approval_id`
2. Admin B calls the same endpoint with `{ approval_id }` → adds signature; if threshold met, the operation executes
3. Approvals expire after the configured window; duplicate signatures by the same admin are rejected (HTTP 409)

Safety properties:

- Token-based auth (no wallet address) is blocked from multi-sig participation
- Request parameters are snapshot at creation time to prevent tampering between approval steps
- Route parameters (e.g., `:id`) are preserved for deferred execution
- All multi-sig events are audit-logged (creation, approval, rejection, expiry)
- Expired approvals are pruned automatically every 60 seconds

4.2 Admin Approval Management

Three dedicated endpoints enable operational oversight:

Endpoint	Method	Description
`/admin/approvals`	GET	List all approvals (filterable by status)
`/admin/approvals/pending`	GET	List pending approvals only
`/admin/approvals/:id/reject`	POST	Reject a pending approval with reason

Data Layer Security

5.1 Current Architecture (Development)

The application currently uses **sql.js** (SQLite via WebAssembly) for data persistence. This is appropriate for development and testing.

Current protections:

- All queries use parameterized statements (prepare/bind pattern)
- Auto-save interval minimizes data loss window
- JSON data fields are properly serialized/deserialized

5.2 Production Migration Plan

A PostgreSQL migration plan is documented in `ops-compliance-plan.md` :

Aspect	Current (Dev)	Target (Production)

Database	sql.js (SQLite)	PostgreSQL 15+
Scaling	Single process	Connection pooling
Durability	In-memory + file	WAL + streaming replication
Backup	File copy	pg_dump + PITR

External Service Resilience

6.1 Arweave Gateway

RWA credential NFT delivery depends on fetching metadata from Arweave. The system uses a **multi-gateway racing strategy**:

- Primary: Concurrent `Promise.any()` across `arweave.net` and `ar-io.net` (15s timeout each)
- Fallback: Sequential retry with 30s timeout per gateway
- Failure: Clear error with gateway identification for debugging

This eliminates single-point-of-failure risk for credential delivery.

6.2 On-Chain Attestation

The release attestation system includes **idempotency protection**:

- Before submitting an attestation, the system queries `getAttestation()` on-chain
- If a RELEASE attestation already exists, the submission is skipped (no wasted gas, no `ReleaseIsFinal` revert)
- This protects against duplicate attestations from retry mechanisms

6.3 Delivery Pipeline

The RWA credential delivery pipeline is protected against concurrent execution:

- A **process-level lock** (`Set`) ensures only one finalizer processes a given trigger at a time
- A **1.5-second stagger** between recipient deliveries prevents rate-limiting from external APIs
- The trigger status is re-read from DB before processing to avoid stale-state race conditions

---

Smart Contract Security

7.1 Contract Architecture

Contract	Function	Key Security Feature
`ReleaseAttestation.sol`	Records on-chain release decisions	`ReleaseIsFinal` — one-shot immutability
`YaultVault.sol`	ERC-4626 yield vault	Standard OpenZeppelin implementation
`VaultShareEscrow.sol`	Per-recipient share escrow	Claim-after-release pattern
`YaultPathClaim.sol`	USDC path-based claims	Pool-based distribution

7.2 Assessment Notes

- **Static review** of contract logic, access control, and error handling has been performed
- **No formal verification, symbolic execution, or fuzz testing** has been conducted
- OpenZeppelin base contracts provide battle-tested implementations
- **Formal third-party audit is strongly recommended before mainnet with significant TVL**

Operational Controls

8.1 Security Configuration

Configuration	Setting
`ADMIN_TOKEN`	Required; no fallback/default value
`ADMIN_WALLETS`	Comma-separated EVM addresses
`MULTISIG_DISABLED`	Only available in non-production env
`MULTISIG_REQUIRED_<action>`	Per-action override for approval threshold
Rate limiter config	Per-endpoint limits enforced at server level

8.2 Audit Trail

Multi-sig operations produce audit records:

Event	Record Type
Approval created	`MULTISIG_APPROVAL_CREATED`
Approval threshold met	`MULTISIG_APPROVAL_EXECUTED`
Approval rejected	`MULTISIG_APPROVAL_REJECTED`

Areas for Improvement

These items do not represent exploitable vulnerabilities but are recommended improvements for production hardening:

9.1 Admin Auth Module Consolidation (Low)

Admin authentication logic (ADMIN_WALLETS parsing, authorization checks) is currently duplicated between `server/api/admin/index.js` and `server/api/authority/verify.js`. This should be extracted into a shared `server/middleware/adminAuth.js` module to eliminate maintenance risk.

9.2 Database Engine (Informational — Migration Planned)

The current sql.js (SQLite WASM) engine is suitable for development but has production limitations: single-process only, in-memory data loss window, and synchronous WebAssembly execution. PostgreSQL migration is planned and documented.

9.3 Relayer Key Management (Informational — HSM Migration Planned)

The relayer private key for on-chain attestation submission is stored as an environment variable. For production, this should be migrated to AWS KMS or GCP Cloud HSM. The relayer wallet ETH balance should be limited to 1–2 days of estimated gas expenditure.

9.4 Finalizer Architecture (Informational)

Two independent background processes (cooldown-finalizer at 30s, scheduler at 60s) both invoke the finalization function. While the process-level lock prevents race conditions within a single instance, this should be consolidated into a single scheduler with distributed locking for multi-instance deployment.

Pre-Launch Recommendations

Priority 1 — Before Mainnet

#	Recommendation	Detail
1	**Formal Smart Contract Audit**	Engage Cyfrin, Trail of Bits, or OpenZeppelin for formal verification and fuzz testing
2	**PostgreSQL Migration**	Complete the documented migration from sql.js to PostgreSQL
3	**HSM Key Management**	Migrate relayer private key to AWS KMS / GCP Cloud HSM
4	**Penetration Testing**	Professional pentest against deployed backend API

Priority 2 — Post-Launch Hardening

#	Recommendation	Detail
5	**Admin Auth Consolidation**	Refactor duplicated admin auth logic into shared middleware
6	**Distributed Locking**	Redis-based or DB advisory locks for multi-instance finalizer
7	**Monitoring & Alerting**	Failed attestations, Arweave timeouts, rate limit breaches
8	**Circuit Breaker Pattern**	Circuit breakers for external service calls (Arweave, mint API, RPC)

Priority 3 — Continuous Improvement

#	Recommendation	Detail
9	**Automated Security Scanning**	Slither (Solidity) + Semgrep (Node.js) in CI/CD pipeline
10	**Incident Response Plan**	Documented SOP for key compromise, contract exploits, data breach

---

Scope Limitations & Disclaimer

1. This assessment was performed by an AI assistant (Claude, Anthropic) through interactive code review, runtime debugging, and architecture analysis during February 2026.
2. Smart contracts were reviewed for logic errors, access control, and custom error handling. **No formal verification, symbolic execution, or fuzz testing was performed.**
3. The frontend application (`webapp/`) was **not in scope**.
4. **No formal penetration testing** was performed against the deployed system. All findings were identified through static analysis and runtime observation.
5. Third-party dependencies (npm packages, OpenZeppelin contracts) were assessed only at their integration points.
6. **This report does NOT constitute a professional third-party security audit.** It is an AI-assisted assessment intended to supplement — not replace — a formal audit by a specialized security firm. A professional audit is **strongly recommended** before mainnet launch with significant TVL.
7. The findings in this report reflect the codebase state as of the assessment date. Subsequent changes may affect the security posture described herein.

Report generated 2026-02-26 by Claude (Anthropic) — AI-assisted security assessment.